

2025-12-23

Low power debug sop

姓名 徐铭骏

掌锐电子系统架构部

目录

Suspend/Resume 简介 2

1. 休眠失败 debug	3
1.1 如系统灭屏后无法进入休眠(即没有 suspend entry 打印)	3
1.2 如果系统已经进入 kernel 休眠流程(suspend entry):	4
2. kernel suspend / resume 各阶段耗时优化	8
3. 休眠成功，休眠电流偏大	9
软件方面 10	
3.1 先确认休眠期间的状态，是否有频繁唤醒	10
3.2 检查是否有未关闭的外设耗电	11
3.3 dump gpio 状态	11
3.4 打印 block LPM 信息	12
3.5 进入休眠前打印 enable clk 信息	13
硬件方面: 15	
3.6 硬件 breakdown，量测电流	15

Suspend/Resume 简介

在 MT8676 平台上, Suspend 即 Suspend to RAM, 此状态使所有的设备进入低功耗状态, 仅保留 RAM 自刷新。

Suspend Power State:

- Tasks “freezed”
- 外部设备/内部部分模块关电或者进入低功耗模式
- System PLL/clock close
- ARM off
- DRAM 自刷新
- PMIC 进入低功耗模式
- VCORE off
- SPM run
- 等待硬件唤醒事件

在 MT8676 Hypervisor (L+L+A) 系统上, Host OS SoS 与 Guest OS Tbox 系统都是 Yocto, Guest OS IVI 系统是 Android。系统级 Suspend 由 Host OS 发起, 触发 Guest OS 运行 Suspend Flow, Guest OS 都完成 Suspend Flow 之后, Host OS 可以进入 Suspend, Host OS Suspend 完成之后整个系统才能进入省电的深度休眠模式。

1. 休眠失败 debug

1.1 如系统灭屏后无法进入休眠(即没有 suspend entry 打印)

可以用以下命令判断是哪个wakelock阻止系统进入休眠:

查看哪些模块持锁:

① `cat /sys/kernel/debug/wakeup_sources`

观察输出结果的第5列active_since, 数字不为0且一直在增大的就是阻止系统进入休眠的wakelock。

name	active_count	event_count	wakeup_count	expire_count	active_since	total_time	max_time	last_change	prevent_suspend_time
event2	0	0	0	0	0	0	0	0	0
event3	0	0	0	0	0	0	0	0	0
event1	0	0	0	0	0	0	0	0	0
notify	88	88	0	0	0	34	1	1196057	0
eventpoll	2	2	0	0	0	0	0	38205	0
timerfd	0	0	0	0	0	1	0	1196057	0
timerfd	0	0	0	0	0	0	0	0	0

② 亦可使用下面的command查看, 输出的那些就是阻止进休眠的wakelock

```
cat /sys/kernel/debug/wakeup_sources | awk -F'\t' '{ if(NR==1) print $1"\t" $7; else if($10) print $1"\t"$10;}'
```

③ 查看 kernel log, 搜索关键字

Pending Wakeup Sources: PowerManagerService disp_crtc2_wakelock

disp_crtc1_wakelock disp_crtc0_wakelock

Pending Wakeup Sources: PowerManagerService disp_crtc2_wakelock

disp_crtc1_wakelock disp_crtc0_wakelock

1.2 如果系统已经进入 kernel 休眠流程(suspend entry):

kernel 休眠成功会有如下 log 打印, 如果没有看到就说明 kernel suspend 没有成功: 需要查看没有 suspend 成功的具体原因。

[2025-10-23 14:49:06] [3533.938080][T1402126] lifecycle: PM: [name:main&][NOBO-MTK-BSP-SYSTEM-STR] noirq suspend of devices complete after 49.035 msecs

[2025-10-23 14:49:06] [3533.939605][T1700000] swapper/7:

[name:mtk_lpm_plat_v1&][name:spm&][lpm_suspend_s2idle_prompt:124] - **suspend enter**

① wakelock

ex:

<6>[426.823002]<1>(0)[1895:system server]PM: **Wakeup pending, aborting**

suspend

<6>[426.823021]<1>(0)[1895:system server]**active wakeup source:leds wakelock**

找申请 leds wakelock 的模块即可, 基本上就是这个模块导致。

② free task timeout

```
<3>[ 852.415255][T1102726] sleep_manager: [name:process6]Freezing of tasks failed after 20.010 seconds (1 tasks refusing to freeze, wq_busy=0):
<6>[ 852.417892][T1102726] sleep_manager: [name:oom_kill6]OOM killer enabled.
<6>[ 852.417959][T1102726] sleep_manager: [name:process6]Restarting tasks ...
```

a. 确认这俩config有开

zcat /proc/config.gz | grep CONFIG_PM_DEBUG

zcat /proc/config.gz | grep CONFIG_PM_SLEEP_DEBUG

b. 输入下面command

echo 1 > /sys/power/pm_debug_messages

c. 重新复现

d. 哪个task 拒绝freeze, 就可以打印出来

ex:

```

80710 [20:29:16:008] [ 221.913438] [T10000002426] sleepmanager: Freezing user space processes failed after 20.007 seconds (1 tasks refusing to freeze, wq_busy=0):
80711 [20:29:16:008] [ 221.913438] [T10000002426] sleepmanager: task:modprobe states:D stack:0 pidi:2405 ppid:2396 flags:0x00000209
80712 [20:29:16:008] [ 221.914578] [T10000002426] sleepmanager: Call trace:
80713 [20:29:16:008] [ 221.917208] [T10000002426] sleepmanager: __switch_to+0x180/0x364
80714 [20:29:16:008] [ 221.918000] [T10000002426] sleepmanager: __schedule+0x158/0x990
80715 [20:29:16:008] [ 221.918742] [T10000002426] sleepmanager: schedule+0x75/0x80
80716 [20:29:16:008] [ 221.919818] [T10000002426] sleepmanager: schedule_timeout+0x80/0x164
80717 [20:29:16:008] [ 221.920329] [T10000002426] sleepmanager: do_wait_for_common+0x108/0x1b0
80718 [20:29:16:008] [ 221.921179] [T10000002426] sleepmanager: wait_for_completion_timeout+0x45/0x6c
80719 [20:29:16:008] [ 221.923106] [T10000002426] sleepmanager: mtk_i2c_transfer+0x5f0/0x1b28 [i2c_xxx] 140000003000000474e550094cbd1f11879670
80720 [20:29:16:010] [ 221.923630] [T10000002426] sleepmanager: mtk_i2c_transfer+0x250/0x40c [i2c_xxx] 140000003000000474e550075e117137283b11
80721 [20:29:16:010] [ 221.925049] [T10000002426] sleepmanager: __i2c_transfer+0x26c/0x72c
80722 [20:29:16:010] [ 221.925879] [T10000002426] sleepmanager: i2c_transfer+0x30/0x130
80723 [20:29:16:010] [ 221.926640] [T10000002426] sleepmanager: gtp_i2c_read+0x1b5/0x328 [gtp_xxx touch] 140000003000000474e550094cbd1f11879670
80724 [20:29:16:011] [ 221.928149] [T10000002426] sleepmanager: gtp_i2c_read_db1_check+0x70/0x212 [gtp_xxx touch] 140000003000000474e550094cbd1f11879670
80725 [20:29:16:011] [ 221.929707] [T10000002426] sleepmanager: gtp_probe+0x600/0x6c4 [gtp_xxx touch] 140000003000000474e550094cbd1f11879670
80726 [20:29:16:011] [ 221.931126] [T10000002426] sleepmanager: i2c_device_probe+0x1e5/0x2c0
80727 [20:29:16:011] [ 221.931994] [T10000002426] sleepmanager: really_probe+0x114/0x480
80728 [20:29:16:011] [ 221.932774] [T10000002426] sleepmanager: driver_probe_device+0xa0/0x160
80729 [20:29:16:012] [ 221.933654] [T10000002426] sleepmanager: driver_probe_device+0x3c/0x1a8
80730 [20:29:16:012] [ 221.934498] [T10000002426] sleepmanager: __driver_attach+0x80/0x1e4
80731 [20:29:16:012] [ 221.935305] [T10000002426] sleepmanager: bus_for_each_dev+0x104/0x160
80732 [20:29:16:012] [ 221.936162] [T10000002426] sleepmanager: driver_attach+0x24/0x34
80733 [20:29:16:012] [ 221.936936] [T10000002426] sleepmanager: bus_add_driver+0x14c/0x294
80734 [20:29:16:012] [ 221.937742] [T10000002426] sleepmanager: driver_register+0x60/0x104
80735 [20:29:16:013] [ 221.938549] [T10000002426] sleepmanager: i2c_register_driver+0x49/0x80
80736 [20:29:16:013] [ 221.939376] [T10000002426] sleepmanager: init_module+0x2c/0x6fc [gtp_xxx touch] 140000003000000474e550094cbd1f11879670
80737 [20:29:16:013] [ 221.940746] [T10000002426] sleepmanager: do_one_initcall+0x58/0x36c
80738 [20:29:16:013] [ 221.941597] [T10000002426] sleepmanager: do_init_module+0x58/0x214
80739 [20:29:16:014] [ 221.942322] [T10000002426] sleepmanager: load_module+0x100/0x1214
80740 [20:29:16:014] [ 221.943103] [T10000002426] sleepmanager: __acpi_sys_fini+0x100/0x330
80741 [20:29:16:014] [ 221.944015] [T10000002426] sleepmanager: invoke_syscall+0x58/0x114
80742 [20:29:16:014] [ 221.944800] [T10000002426] sleepmanager: ei0_svc_common+0x50/0x8c
80743 [20:29:16:014] [ 221.945570] [T10000002426] sleepmanager: do_ei0_svc+0x10/0x28
80744 [20:29:16:015] [ 221.946298] [T10000002426] sleepmanager: ei0_svc+0x30/0x60
80745 [20:29:16:015] [ 221.946992] [T10000002426] sleepmanager: ei0t_64_sync_handler+0x70/0x8c
80746 [20:29:16:015] [ 221.947837] [T10000002426] sleepmanager: ei0t_64_sync+0x1a8/0x1aa
80747 [20:29:16:015] [ 221.948767] [T10000002426] sleepmanager: OOM killer enabled.
80748 [20:29:16:015] [ 221.949471] [T10000002426] sleepmanager: Restarting tasks ...
80749 [20:29:16:016] [ 221.950272] [T10000000074] kworker/u16:3 [name:apna] [SPM] mcuysys was not off
80750 [20:29:16:016] [ 221.952440] [T10000002426] done.

```

③ dpm suspend阶段，有device suspend fail，return NOTIFY_BAD 这种也会中止休眠流程，导致休眠失败

ex:

```
int __maybe_unused mtk_ingsys_pm_suspend(struct device *dev)
{
    struct mtk_ingsys_dev *ingsys_dev = dev_get_drvdata(dev);
    int ret, num;
    bool idle = true;

    ret = wait_event_timeout
        (ingsys_dev->flushing_waitq, (idle = mtk_ingsys_idle(dev)),
        msecs_to_jiffies(1000 / 30 * DIP_COMPOSING_MAX_NUM * 3));
    num = atomic_read(&ingsys_dev->num_composing);
    if (!ret && !idle) {
        dev_info(dev, "%s: flushing SCP job timeout, num(%d), idle(%d)\n",
            __func__, num, idle);

        return NOTIFY_BAD;
    }

#ifdef NEED_PM
    if (pm_runtime_suspended(dev)) {
        dev_info(dev, "%s: pm_runtime_suspended is true, no action\n",
            __func__);
        return NOTIFY_DONE;
    }

    ret = pm_runtime_put_sync(dev);
    if (ret) {
        dev_info(dev, "%s: pm_runtime_put_sync failed: (%d)\n",
            __func__, ret);
        return NOTIFY_BAD;
    }
#endif
    return NOTIFY_DONE;
}
```

④ 休眠时屏幕被打开，没有关闭，导致休眠失败

```
<6>[68119.465149][ T2030] PM: suspend entry (s2idle)
<6>[68119.505043][ T2030] Filesystems sync: 0.039 seconds
<6>[68119.505694][ T2030] [DISP]Disabling CRTC wakelock
<6>[68119.505701][ T2030] [DISP]mtk_drm_sys_suspend crtc active!! wake_state:0x7
<5>[68119.505704][ T2030] [MDP]cmdq_core_resume_notifier ref:0
<5>[68119.505707][ T2030] [cmdq] mdp buffer pool already created
<6>[68119.505748][ T2030] Resume cause unknown
<6>[68119.505752][ T2030] PM: suspend exit
```

```
if (!wake_state) {
    DDPMMSG("%s+++\\n", __func__);
    goto OUT;
} else {
    //not support suspend when crtc is active
    DDPMMSG("%s crtc active!! wake_state:0x%x\\n", __func__, wake_state);
    return -EBUSY;
}
```

⑤ 同理，如果有类似于屏幕没关，camera 没关就开始进 kernel 休眠的情况，可以在对应 module 中的 notifier callback 去判断下，并返回 NOTIFY_BAD。

⑥ 如果需要打印更多休眠唤醒 log：
可以在对应 os 中打开下面两个 config，

CONFIG_PM_DEBUG=y

CONFIG_PM_SLEEP_DEBUG=y

并在对应 os 输入：

echo 0 > /sys/module/printk/parameters/console_suspend

echo 1 > /sys/module/kernel/parameters/initcall_debug

echo 1 > /sys/power/pm_print_times

echo 8 > /proc/sys/kernel/printk

echo 1 > /sys/power/pm_debug_messages

echo 1 > /proc/mprintk //如果要抓串口 log，可以打开：Mobilelog 会默认打开

2. kernel suspend / resume 各阶段耗时优化

休眠唤醒的速度也会对平均功耗产生一定的影响，所以需要尽可能的优化到最短时间

1. 休眠和唤醒可以大致划分为下面几个阶段，在 kernel log 中搜索下面的关键字，可以大致得到每个阶段的耗时，针对耗时异常的阶段再进行细化分析

休眠:

suspend entry

Freezing user space processes

Freezing remaining freezable tasks

suspend of devices complete

start suspend of devices complete

late suspend of devices complete

noirq suspend of devices complete

suspend enter

唤醒:

wake up by

noirq resume of devices complete

early resume of devices complete

resume of devices complete

Restarting tasks

suspend exit

2. 可以在各个os中打开下面两个config,

CONFIG_PM_DEBUG=y

CONFIG_PM_SLEEP_DEBUG=y

并输入:

echo 0 > /sys/module/printk/parameters/console_suspend

echo 1 > /sys/module/kernel/parameters/initcall_debug

echo 1 > /sys/power/pm_print_times

echo 8 > /proc/sys/kernel/printk

echo 1 > /sys/power/pm_debug_messages

echo 1 > /proc/mtprintk //如果要抓串口log, 可以打开: Mobilelog会默认打开

会把各个 device, driver 的休眠唤醒耗时打印出来。(打开 config 后, 打印的 log 会很多, 也可能会增加休眠唤醒的时间, 可以重点看下相对耗时较长的阶段, device) 进行优化。

3. 休眠成功，休眠电流偏大

Kernel suspend 完成后，SPM(System Power Manage，用于电源管理的 MCU)将接管整个系统的资源，控制 DRAM 进入自刷新模式，关闭 26M 和 VCORE，通知 PMIC 进入 Low Power Mode。至此，整个休眠流程完成，系统处于低功耗状态，等待唤醒事件发生。

SPM 掌控着 kernel suspend 之后系统是否能掉到最小电流的关键逻辑，你可以把它理解成一个投票机制当系统的关键资源(memory、clock)没有任何人使用的时候，它就会让系统进入一个真正的深睡状态(最小电流)只要它检测到有任何资源请求还没释放，系统就无法降到底电所以在底电问题上的 debug 流程中，我们不仅仅要看 kernel 有没有 suspend 成功，还要看 SPM 的状态是否正确。

跟 SPM 强相关的一个东西就是系统中的时钟请求信号，也就是 26M 时钟开关的控制逻辑；因为系统工作在最小电流的时候，SPM 只依靠 32K 时钟工作；因此要判断系统是不是已经到深睡状态，就要看 26M 有没有关闭。

SPM 会去关注各个子系统(modem, connectivity 等)的状态，请求 26M 的信息会记录在 SPM 的寄存器里面，这个就是我们通过 log 排查的依据。

如果我们看到唤醒后打印的休眠信息中 **26M_off_pct = 0, vcore_off_pct = 0**，就要查看下是谁拉住了 request，导致 26M, vcore 没有关闭。

```
suspend wake up by R12_EINT_EVENT_B, timer_out = 1493413, r13 = 0x80001ac0,
debug_flag = 0xd00430fc 0x20f806f, r12 = 0x40, r12_ext = 0x0, raw_sta = 0x0 0x0 0x0,
idle_sta = 0x0, req_sta = 0x0 0x0 0x0 0x0 | 0x78007c 0x0 0x60000000 0x181819c |
0x0 0x0 0x1bc0 0x0, cg_check_sta = 0x0, isr = 0x0, rt_req_sta0 = 0x0 rt_req_sta1 =
0xb71b0 rt_req_sta2 = 0x0 rt_req_sta3 = 0x7068605c dram_sw_con_3 = 0x0,
raw_ext_sta = 0x848a955, wake_misc = 0x180030, pcm_flag = 0x2280060 0x180
0x2200060 0x180, req = 0x18f60000, clk_settle = 0x60fe, debug_spare_5 = 0x0,
debug_spare_6 = 0x10000000, wlk_cntcv_l = 0xb5f12d73, wlk_cntcv_h = 0x1,
26M_off_pct = 0, vcore_off_pct = 0
```

软件方面

3.1 先确认休眠期间的状态，是否有频繁唤醒

搜索关键字: `suspend entry|suspend exit|suspend wake up by|suspended for`

`suspend wake up by`: 打印唤醒源

`suspended for`: 打印的是休眠时间

如果有，判断唤醒是否正常

(如果需要更多关键字可以搜索: `suspended for|pm: suspend| suspend wake up by
|suspend enter|PM: Pending Wakeup Sources|Wakeup Source|Restarting
tasks|Freezing user space processes|Syncing filesystem|complete after|aborted
after|failed after|Resume caused by |is pending|eint_mask|wakeuptd by|Resume
cause
)`

常见唤醒源:

Name	Control Bit	Description	Can it be Disabled
R12_PCM_TIMER	0	设定时器，超时后唤醒系统	Y
R12_KP_IRQ_B	2	检测到键盘按下/释放动作	Y
R12_APWDT_EVENT_B	3	RGU 在 suspend mode 中唤醒 CPU	Y
R12_APXGPT1_EVENT_B	4	AP GPT 计时器超时事件	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC (Wi-Fi/BT/GPS) 唤醒系统	Y
R12_EINT_EVENT_B	6	EINT 事件唤醒系统	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout 唤醒系统	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF 唤醒事件	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF 唤醒事件	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM 相关唤醒系统	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor 相关唤醒系统	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP 相关唤醒系统	Y
R12_USBX_CDSC_B	14	USB 相关唤醒系统	Y
R12_USBX_POWERDOWN_B	15	USB 远程唤醒	Y
R12_SYS_TIMER_EVENT_B	18	System timer 唤醒系统	N
R12_EINT_EVENT_SECURE_B	19	EINT event 唤醒系统	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ 唤醒事件	Y
R12_MD2AP_PEER_EVENT_B	23	MD 相关唤醒系统	Y
R12_MD1_WDT_B	25	MD1 看门狗超时	Y
R12_REG_CPU_WAKEUP_B	28	内部唤醒源	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS 的唤醒源	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe 相关唤醒源	Y
R12_MSDC_WAKEUP_EVENT_B	31	MSDC 相关唤醒源	Y

3.2 检查是否有未关闭的外设耗电

3.3 dump gpio 状态

```
echo gpio_dump 1 > /proc/mtk_lpm/suspend/suspend_state // spm 打印 gpio 信息
```

可从 gpio 的状态是否符合预期来定位影响功耗的 module

3.4 打印 block LPM 信息

```
echo 1 > /proc/mtk_lpm/spm/block_threshold
```

Ex:

```
① suspend warning:(OneShot) System LPM is blocked by spm
② suspend warning:(OneShot) System LPM is blocked by md conn
③ suspend warning:(OneShot) System LPM is blocked by md

④ <6>[ 457.243003][T400000] swapper/4:
[name:mtk_lpm_dbg_mt6897&][name:spm&][SPM] suspend wake up by
R12_EINT_EVENT_B, timer_out = 1493413, r13 = 0x80001ac0, debug_flag =
0xd00430fc 0x20f806f, r12 = 0x40, r12_ext = 0x0, raw_sta = 0x0 0x0 0x0, idle_sta = 0x0,
req_sta = 0x0 0x0 0x0 0x0 | 0x78007c 0x0 0x60000000 0x181819c | 0x0 0x0 0x1bc0
0x0, cg_check_sta = 0x0, isr = 0x0, rt_req_sta0 = 0x0 rt_req_sta1 = 0xb71b0 rt_req_sta2
= 0x0 rt_req_sta3 = 0x7068605c dram_sw_con_3 = 0x0, raw_ext_sta = 0x848a955,
wake_misc = 0x180030, pcm_flag = 0x2280060 0x180 0x2200060 0x180, req =
0x18f60000, clk_settle = 0x60fe, debug_spare_5 = 0x0, debug_spare_6 = 0x10000000,
wlk_cntcv_l = 0xb5f12d73, wlk_cntcv_h = 0x1, 26M_off_pct = 0, vcore_off_pct = 0
<6>[ 457.243003][T400000] swapper/4:
[name:mtk_lpm_dbg_common_v2&][name:spm&][SPM] suspend warning:(OneShot)
System LPM is blocked by
<6>[ 457.243003][T400000] swapper/4:
[name:mtk_lpm_dbg_common_v2&][name:spm&][SPM] HWCG sta :[0] 0x0 0x0 0x0 [[1]
0x0 0x18000000 0x0 [[2] 0x0 0x18000000 0x0 [[3] 0x0 0x18000000 0x0 [[4] 0x0
0x18000000 0x10 [[5]
<6>[ 457.243003][T400000] swapper/4:
[name:mtk_lpm_dbg_common_v2&][name:spm&][SPM] PERI_CG sta :[0] 0x0 [[1] 0x0
[[2] 0x0 [[3] 0x0 [[4] 0x0 .
<6>[ 457.243003][T400000] swapper/4:
[name:mtk_lpm_dbg_mt6897&][name:spm&][SPM] Suspended for 45.575 seconds
```

解析: [SPM] HWCG sta :[0] 0x0 0x0 0x0 [[1] 0x0 0x18000000 0x0 [[2] 0x0 0x18000000
0x0 [[3] 0x0 0x18000000 0x0 [[4] 0x0 0x18000000 **0x10** [[5]

是这里的 0x10 代表被 clk 拉住了 request，导致 26M，Vcore 没有被关闭，是 univpll block，最后查到是 vcp 会概率性的多开一次 univpll，导致 suspend 时候 univpll 没有关闭。

⑤ req_sta 的值也需要检查，

Ex:

req_sta 此处栏位的值为 0x3d，代表 md 拉住了 request

```
[name:spm&][SPM] suspend wake up by R12_AP2AP_PEER_WAKEUPEVENT_B,
timer_out = 21, r13 = 0x80001bdc, debug_flag = 0x10800000 0x3, r12 = 0x4000000,
r12_ext = 0x0, raw_sta = 0x0 0x0 0x0, idle_sta = 0x0, req_sta = 0x0 0x0 0x0 0x0 |
0x78007c 0x0 0x60000000 0xe181819c | 0x3d 0x0 0x1bc0 0x0, cg_check_sta = 0x0, isr =
0x0, rt_req_sta0 = 0x0 rt_req_sta1 = 0x10af79e rt_req_sta2 = 0x0 rt_req_sta3 =
0x7068605c dram_sw_con_3 = 0x0, raw_ext_sta = 0x848a955, wake_misc = 0x180030,
pcm_flag = 0x2680060 0x180 0x2600060 0x180, req = 0x18f60000, clk_settle = 0x60fe,
debug_spare_5 = 0x0, debug_spare_6 = 0x10000000, wlk_cntcv_l = 0x4caed0bb,
wlk_cntcv_h = 0x89, 26M_off_pct = 0, vcore_off_pct = 0
```

⑥ 除此之外还有一些只有 MTK 相关 owner 才能解析出来特定 bit 位代表的信息，需要提供 log 给 MTK 具体分析，现有 spm debug 方式，已经可以满足软件方面低功耗 debug 的需求。

3.5 进入休眠前打印 enable clk 信息

打印出 enable clk，可以帮忙直观快速的判断出哪些模块导致了休眠电流偏高；
目前可以做到通过 command 来控制输出，patch 需要整理。

和打印 block 信息共用次 command:

```
echo 1 > /proc/mtk_lpm/spm/block_threshold
```

```
[ 53.361949][T1700000] swapper/7: [name:mtk_lpm_plat_v1&][name:spm&][lpm_suspend_s2idle_prompt:127] - suspend enter
[ 53.361949][T1700000] swapper/7:      clock      enable prepare protect      rate
[ 53.361949][T1700000] swapper/7: -----
[ 53.361949][T1700000] swapper/7: clk13m              1      1      0      13000000
[ 53.361949][T1700000] swapper/7: clk26m              13     13      0      26000000
[ 53.361949][T1700000] swapper/7: ptppll-pil-ctpll      1      1      0      312499878
[ 53.361949][T1700000] swapper/7: armpil-b-pil-ct-b      1      1      0      1199999817
```

[53.361949[T1700000] swapper/7:	armpll-bl-pl-ct-bl	1	1	0	599999909
[53.361949[T1700000] swapper/7:	armpll-bl-pl-ct-bl	1	1	0	799999878
[53.361949[T1700000] swapper/7:	ccipll-pl-ctpl	1	1	0	780000000
[53.361949[T1700000] swapper/7:	emipll	1	1	0	728000000
[53.361949[T1700000] swapper/7:	mpll	1	1	0	208000000
[53.361949[T1700000] swapper/7:	mainpll	3	4	0	2184000000
[53.361949[T1700000] swapper/7:	mainpll_d4_d2	1	1	0	273000000
[53.361949[T1700000] swapper/7:	dpmaif_main_sel	1	1	0	273000000
[53.361949[T1700000] swapper/7:	dpmaif_main_ck	1	1	0	273000000
[53.361949[T1700000] swapper/7:	ifrao_dpmaif_main	1	1	0	273000000
[53.361949[T1700000] swapper/7:	mainpll_d7_d2	2	2	0	156000000
[53.361949[T1700000] swapper/7:	peri_faxi_sel	1	1	0	156000000
[53.361949[T1700000] swapper/7:	peri_faxi_ck	2	2	0	156000000
[53.361949[T1700000] swapper/7:	peraop_audio_mst_ck	1	1	0	156000000
[53.361949[T1700000] swapper/7:	peraop_audio_slv_ck	1	1	0	156000000
[53.361949[T1700000] swapper/7:	axi_sel	1	1	0	156000000
[53.361949[T1700000] swapper/7:	axi_ck	9	9	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif4_md	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif2_md	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif2_ap	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif5_md	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_cldmabolk	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif_md	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif_ap	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif1_md	1	1	0	156000000
[53.361949[T1700000] swapper/7:	ifrao_ccif1_ap	1	1	0	156000000
[53.361949[T1700000] swapper/7:	f26m_ck	2	2	0	26000000
[53.361949[T1700000] swapper/7:	ifrao_dpmaif_26m	1	1	0	26000000
[53.361949[T1700000] swapper/7:	pextpcfg_ao_mac_rclk	2	2	0	26000000
[53.361949[T1700000] swapper/7:	tok_26m_mx9_ck	2	2	0	26000000
[53.361949[T1700000] swapper/7:	aud_intbus_sel	1	1	0	26000000
[53.361949[T1700000] swapper/7:	aud_intbus_ck	1	1	0	26000000
[53.361949[T1700000] swapper/7:	peraop_aud_intbusck	1	1	0	26000000
[53.361949[T1700000] swapper/7:	uart_sel	1	1	0	26000000
[53.361949[T1700000] swapper/7:	uart_ck	1	1	0	26000000
[53.361949[T1700000] swapper/7:	peraop_uart0	1	1	0	26000000

硬件方面：

3.6 硬件 breakdown, 量测电流

如果 26M_off_pct, vcore_off_pct 已经很高了, 达到了 99, 说明软件上已经睡到最深, 漏电行为软件上已经没法去 debug, 需要硬件介入量测是否哪里存在漏电:

根据漏电的情况, 查看对应 module 的休眠状态并进行优化。

```
行 151780: <6>[11615.808112][ T0] [name:spm&][SPM] suspend wake up  
by R12_EINT_EVENT_B, timer_out = 16136723, r13 = 0x80001ac0, debug_flag =  
0xd00433ff0x20fe06f, r12 = 0x40, r12_ext = 0x0, raw_sta = 0x0 0x0 0x0, idle_sta = 0x0,  
req_sta = 0x0 0x0 0x0 0x0 | 0x78007c0x0 0x60000000 0x787879c | 0x0 0x0 0x1b40 0x0,  
cg_check_sta = 0x0, isr = 0x0, rt_req_sta0 = 0x0 rt_req_sta1 = 0xb71b0 rt_req_sta2 =  
0xf6391f rt_req_sta3 = 0x7068605c dram_sw_con_3 = 0x0, raw_ext_sta = 0x848a955,  
wake_misc = 0x190030, pcm_flag = 0x2280060 0x180 0x2200060 0x180, req =  
0x18f60000, clk_settle = 0x60fe, debug_spare_5 = 0x0, debug_spare_6 = 0x10000000,  
wlk_cntcv_l = 0xa7e03957, wlk_cntcv_h = 0x24, 26M_off_pct = 99, vcore_off_pct = 99  
行 151781: <6>[11615.808112][ T0] [name:spm&][SPM] Suspended for 492.453 seconds
```